نسخه‌ی A · README

🏦 چالش «بساز و منتشر کن» — نسخه‌ی A: MiniBank

دوره‌ها: Go — From Beginner to Pro (جلسات ۱ تا ۲۰) + Linux → Go → DevOps Bootcamp (جلسات ۱ تا ۳۲)

زمان: ۳ ساعت · **امتیاز:** ۸۰ ساخت + ۲۰ دفاع حضوری + ۱۵ امتیاز اضافه · **نمره‌ی قبولی:** ۶۰

شیوه: انفرادی، از پوشه‌ی خالی · **ابزار:** همه‌چیز آزاد است، حتی هوش مصنوعی

🎯 مأموریت

یک استارتاپ فین‌تک به یک **سرویس دفتر کل (ledger)** نیاز دارد: یک بانک کوچک که حساب‌ها را نگه می‌دارد، پول را بین آن‌ها جابه‌جا می‌کند و **حتی یک ریال را هم گم نمی‌کند**؛ حتی وقتی صدها درخواست در یک لحظه می‌رسد.

تو باید **MiniBank** را با Go بسازی و بعد مثل یک مهندس DevOps **منتشرش کنی**:

- یک API از نوع REST با Go که داده‌هایش را در **PostgreSQL** نگه می‌دارد
- تست‌های واحد، تست هم‌زمانی و تست HTTP
- اسکریپت‌های Bash برای بررسی محیط و تست دود (smoke test)
- یک ایمیج Docker کوچک و امن
- اجرای کامل با Docker Compose
- یک پایپ‌لاین CI با GitHub Actions
- استقرار روی Kubernetes با **دو رپلیکا**

از یک پوشه‌ی خالی شروع می‌کنی و بعد از ۳ ساعت یک ریپازیتوری Git تحویل می‌دهی.

بعد از آزمون:

۱. مدرس یک **تصحیح‌گر خودکار (auto-grader)** را روی برنامه‌ی در حال اجرای تو و ریپازیتوری‌ات اجرا می‌کند. این برنامه همه‌ی endpointها را صدا می‌زند، ورودی خراب می‌فرستد و **صدها درخواست هم‌زمان** شلیک می‌کند تا ببیند آیا پول‌ها درست جمع می‌شوند یا نه.

۲. یک **دفاع حضوری ۷ تا ۱۲ دقیقه‌ای** بدون هوش مصنوعی انجام می‌دهی.

> 📖 قبل از شروع ساعت آزمون، ۱۰ دقیقه وقت مطالعه داری (بدون نوشتن کد): همین فایل و PLAN.md را کامل بخوان. بعد قدم‌به‌قدم طبق **PLAN.md** جلو برو.

🗺️ این چالش چه چیزی را می‌سنجد

مهارت	دوره‌ی Go	بوت‌کمپ DevOps	مرحله
ساختار پروژه، struct، متد، اشاره‌گر	۱ تا ۹	۷ تا ۱۱	S1
اینترفیس و خطاها (`errors.Is`)	۱۰ و ۱۱	۱۲	S1

مهارت	دوره‌ی Go	بوت‌کمپ DevOps	مرحله
هم‌روندی (`sync.Mutex`، goroutine، `context`)	۱۲ و ۱۳	۱۳	S1 و S2
JSON و سرور HTTP	۱۵ و ۱۷	۱۴	S1
پایگاه داده و تراکنش SQL	۱۸	۱۵	S1
پیکربندی، middleware، خاموشی مرتب	۱۹ و ۲۰	۱۶ و ۲۰	S1
تست‌نویسی	۱۶	۱۴	S2
لینوکس و اسکریپت Bash	–	۱ تا ۳	S3
Git، GitHub و CI	–	۴ تا ۶	S0، S6 و S8
Docker و Docker Compose	۲۰	۲۱ تا ۲۴	S4 و S5
Kubernetes	–	۲۵ تا ۲۹	S7
مانیتورینگ، Ingress و Helm	–	۲۸ و ۲۹	امتیاز اضافه

📜 قوانین آزمون

۱. **انفرادی است.** کمک گرفتن از هر انسانی (هم‌کلاسی، دوست، فروم، گروه تلگرامی) ممنوع است.

۲. **هر ابزاری آزاد است:** مستندات، ریپازیتوری دوره‌ها، موتور جستجو و دستیارهای هوش مصنوعی (ChatGPT، Claude، Gemini، Copilot و …).

۳. **مسئول تک‌تک خط‌ها خودت هستی.** در دفاع باید بتوانی هر بخشی از ریپازیتوری‌ات را توضیح بدهی. طبق قانون **«توضیح بده یا از دست بده»**، اگر بعد از دو سؤال ساده‌تر نتوانی یک مورد امتیازی (یک چک‌باکس در PLAN.md) را توضیح بدهی، امتیاز همان مورد صفر می‌شود؛ چه امتیاز ساخت باشد و چه امتیاز اضافه. جمله‌ی «هوش مصنوعی نوشت» توضیح حساب نمی‌شود.

۴. در README خودت یک بخش **«استفاده از هوش مصنوعی» (AI usage)** بنویس: از چه ابزاری، برای چه کاری، و دست‌کم یک جا که هوش مصنوعی اشتباه کرد و تو درستش کردی. استفاده‌ی صادقانه هیچ اشکالی ندارد؛ پنهان‌کاری دارد: بخش AI usage خالی یا غیرواقعی یعنی از دست دادن ۱ امتیاز S8.

۵. سر هر 🚩 **نقطه‌ی بازرسی** در PLAN.md کامیت کن. هر مرحله را روی **شاخه‌ی (branch) خودش** بساز و بعد در `main` ادغام (merge) کن. تاریخچه‌ی Git نمره دارد و خط زمانی کارت را نشان می‌دهد. یک کامیت غول‌آسا در دقیقه‌ی آخر یعنی از دست دادن امتیاز Git و یک دفاع طولانی‌تر.

۶. **توقف قطعی در ۳:۰۰، یعنی دقیقاً سه ساعت بعد از شروع آزمون.** همه‌ی زمان‌های این اسناد (مثل ۰:۱۵ و ۳:۰۰) از لحظه‌ی شروع آزمون حساب می‌شوند و ساعت شروع روی تخته نوشته می‌شود. فقط کامیت‌هایی حساب می‌شوند که قبل از پایان زمان روی GitHub **push** شده باشند.

۷. **قواعد فنی:**

- از **Go نسخه‌ی ۱.۲۶ یا جدیدتر** استفاده کن. الگوهای مسیریابی مثل `GET /accounts/{id}` از Go 1.22 آمده‌اند، ولی نسخه‌های جدید کتابخانه‌های PostgreSQL و Prometheus به Go 1.25 و 1.26 نیاز دارند. اگر مجبوری با Go قدیمی‌تر کار کنی، نسخه‌های قدیمی‌تر کتابخانه‌ها را بگیر (این‌ها با Go 1.22 کار می‌کنند): `go get github.com/jackc/pgx/v5@v5.7.4 github.com/prometheus/client_golang@v1.22.0`

- لایه‌ی HTTP فقط با کتابخانه‌ی استاندارد `net/http` نوشته شود. فریم‌ورک وب (gin، echo، fiber، chi) ممنوع است.
- ماژول‌های مجاز: درایور PostgreSQL (پیشنهاد: `github.com/jackc/pgx/v5`) و فقط برای امتیاز اضافه‌ی B1، `github.com/prometheus/client_golang`. بقیه‌ی کد فقط با کتابخانه‌ی استاندارد.

۸. **آماده‌سازی سیستم** باید از قبل، طبق «چک‌لیست آماده‌سازی» که یک هفته قبل از آزمون برایت فرستاده شد، انجام شده باشد (خلاصه‌اش پایین همین فایل آمده است). مشکل نصب در روز آزمون دلیل وقت اضافه نیست.

📦 چه چیزی تحویل می‌دهی

یک ریپازیتوری GitHub: عمومی، یا خصوصی با دعوت از حساب GitHub مدرس (Settings → Collaborators؛ نام کاربری مدرس روز آزمون روی تخته نوشته می‌شود). آدرس ریپازیتوری را تا ۱۵:۰ به روشی که مدرس اعلام می‌کند تحویل بده. ساختار پیشنهادی:

```
minibank/
├── cmd/minibank/main.go        # سیم‌کشی: config، انتخاب store، سرور، خاموشی مرتب
├── internal/
│   ├── bank/                   # Account، Transaction، اعتبارسنجی، خطاها (+ تست)
│   ├── store/                  # اینترفیس Store + memory + postgres (+ تست)
│   └── api/                    # handlerها، JSON، نگاشت خطا به status، middleware (+ تست)
├── scripts/
│   ├── check-env.sh
│   └── smoke.sh
├── Dockerfile
├── .dockerignore
├── compose.yaml
├── .env.example
├── .github/workflows/ci.yml
├── k8s/                        # namespace، configmap، secret، postgres، app
├── PLAN.md                     # همین نقشه‌ی راه، با بخش «تصمیم‌های من» پر شده
└── README.md                   # خودت: معرفی، نحوه‌ی اجرا، بخش AI usage README
```

هر ساختار تمیز دیگری هم قبول است، به شرط این‌که **دست‌کم سه پکیج** جدا داشته باشد (دامنه، ذخیره‌سازی، HTTP).

📐 قرارداد API (به‌صورت خودکار تصحیح می‌شود)

آدرس پایه: `http://localhost:8080` (پورت با متغیر `PORT` عوض می‌شود)

هر پاسخی که handlerهای تو می‌نویسند (حتی پاسخ خطا) باید هدر `Content-Type: application/json` داشته باشد.

> ⚠️ تصحیح‌گر فقط به همین قرارداد نگاه می‌کند. نام اشتباه یک فیلد (مثلاً `Owner` به‌جای `owner`) یا status code اشتباه (مثلاً `200` به‌جای `201`) یعنی از دست دادن امتیاز همان بررسی.

شکل داده‌ها

حساب (Account):

```
{ "id": 1, "owner": "Sara", "balance": 1500, "created_at": "2026-10-20T09:30:00Z" }
```

تراکنش (Transaction):

```
{ "id": 7, "type": "transfer", "from": 1, "to": 2, "amount": 300, "created_at": "2026-10-20T09:41:12Z" }
```

- مقدار `type` یکی از این سه است: `"deposit"` (واریز)، `"withdraw"` (برداشت)، `"transfer"` (انتقال).
- واریز فقط `to` دارد، برداشت فقط `from` دارد و انتقال هر دو را دارد. کلیدی که وجود ندارد **کلاً در JSON نباید** (نه `null` و نه `0`). راهنمایی: تگ `omitempty` . اگر در PostgreSQL، `from_id` یا `to_id` را `NULL` ذخیره می‌کنی، موقع خواندن از `COALESCE(from_id, 0)` (یا `*int64`) استفاده کن؛ pgx نمی‌تواند `NULL` را در `int64` بریزد و پاسخ `500` می‌شود.
- شناسه‌ها (`id`) عدد صحیح مثبتی هستند که سرور تعیین می‌کند. شناسه‌ها صعودی‌اند و هرگز دوباره استفاده نمی‌شوند. لازم نیست از ۱ شروع شوند.
- شناسه‌ها `int64` هستند. در PostgreSQL ستون‌های `id` ، `from_id` ، `to_id` و `balance` را `BIGINT` بگیر (مثلاً `BIGINT GENERATED ALWAYS AS IDENTITY` یا `BIGSERIAL`)، نه `SERIAL` یا `INTEGER` . تصحیح‌گر برای «حساب ناموجود» شناسه‌ای مثل `999999999999` می‌فرستد و جواب باید `404` باشد، نه `500` .
- فیلد `created_at` با فرمت RFC 3339 است (UTC پیشنهاد می‌شود).

خطا:

```
{ "error": "insufficient funds" }
```

قوانین پول 💰

- پول یک **عدد صحیح** (`int64`) است؛ مثلاً به ریال یا تومان. **هرگز از `float64` برای پول استفاده نکن.**
- مقدار معتبر `amount` یک عدد صحیح JSON است که **بزرگ‌تر از ۰ و حداکثر** `1000000000` (یک میلیارد) باشد. یعنی `0` ، عدد منفی، `10.5` ، رشته‌ی `"100"` ، نبودن فیلد یا عدد بزرگ‌تر از یک میلیارد، همه خطای `400` هستند.
- موجودی هیچ حسابی **هرگز** نباید زیر صفر برود (در دیتابیس هم با `CHECK (balance >= 0)` از آن محافظت کن).
- انتقال پول **اتمیک (atomic)** است: یا موجودی هر دو حساب تغییر می‌کند و ردیف تراکنش ثبت می‌شود، یا هیچ اتفاقی نمی‌افتد.
- نام صاحب حساب (`owner`) با `strings.TrimSpace` تمیز می‌شود و باید **۲ تا ۴۰ کاراکتر باشد، شمرده‌شده بر حسب rune و نه بایت.** مثلاً `"سارا"` معتبر است (۴ rune ولی ۸ بایت) و `"س"` نامعتبر است (۱ rune ولی ۲ بایت). پاسخ، نام تمیزشده را برمی‌گرداند.

Endpointها

#	متد و مسیر	بدنه	پاسخ موفق	خطاها
۱	`GET /healthz`	–	`200` و `{"status":"ok"}` (هرگز به دیتابیس دست نمی‌زند)	–
۲	`GET /readyz`	–	`200` و `{"status":"ready"}` وقتی Ping دیتابیس موفق است	`503` و `{"status":"not ready"}`
۳	`POST /accounts`	`{"owner":"Sara"}`	`201` + Account (موجودی ۰)	`400`

#	متد و مسیر	بدنه	پاسخ موفق	خطاها
۴	`GET /accounts`	–	`200` + آرایه‌ی Accountها به ترتیب صعودی `id` ؛ اگر خالی بود `[]` (هرگز `null`)	–
۵	`GET /accounts/{id}`	–	Account + `200`	`400` ، `404`
۶	`POST /accounts/{id}/deposit`	`{"amount":500}`	Account به‌روزشده + `200`	`400` ، `404`
۷	`POST /accounts/{id}/withdraw`	`{"amount":200}`	Account به‌روزشده + `200`	`400` ، `404` ، `409`
۸	`POST /transfers`	`{"from":1,"to":2,"amount":300}`	Transaction + `201`	`400` ، `404` ، `409`
۹	`GET /accounts/{id}/transactions`	–	`200` + آرایه‌ی تراکنش‌هایی که `from` یا `to` آن‌ها این حساب است، **قدیمی‌ترین اول**؛ اگر نبود `[]`	`400` ، `404`

معنی و ترتیب خطاها

کد	چه زمانی
`400`	JSON خراب یا نوع اشتباه، `amount` نامعتبر، `owner` نامعتبر، `id` مسیر عدد صحیح مثبت نیست (مثل `abc` یا `0`)، در انتقال `from` یا `to` عدد صحیح مثبت نیست، یا `from == to`
`404`	حساب پیدا نشد (هر کدام از دو حساب در انتقال)
`409`	موجودی کافی نیست: `{"error":"insufficient funds"}` و موجودی بدون تغییر می‌ماند

اگر یک درخواست چند مشکل داشت، **این ترتیب** رعایت شود: اول `400` ، بعد `404` ، بعد `409` .

پیشنهاد: حجم بدنه‌ی درخواست را با `http.MaxBytesReader` به ۱ مگابایت محدود کن (بدنه‌ی بزرگ‌تر باید خطای `4xx` بگیرد، مثلاً `413` یا `400` ، نه `500` ؛ این مورد نمره ندارد). اگر متد اشتباه بود (مثلاً `DELETE /accounts`)، خود `ServeMux` جواب `405` می‌دهد.

یک نمونه‌ی کامل

```
curl -s -X POST localhost:8080/accounts -d '{"owner":"Sara"}'
# 201 {"id":1,"owner":"Sara","balance":0,"created_at":"2026-10-20T09:30:00Z"}

curl -s -X POST localhost:8080/accounts -d '{"owner":"Reza"}'
# 201 {"id":2,"owner":"Reza","balance":0,...}

curl -s -X POST localhost:8080/accounts/1/deposit -d '{"amount":1000}'
# 200 {"id":1,"owner":"Sara","balance":1000,...}

curl -s -X POST localhost:8080/transfers -d '{"from":1,"to":2,"amount":300}'
# 201 {"id":2,"type":"transfer","from":1,"to":2,"amount":300,"created_at":"..."}

curl -s -X POST localhost:8080/accounts/2/withdraw -d '{"amount":5000}'
# 409 {"error":"insufficient funds"}

curl -s localhost:8080/accounts/1/transactions
# 200 [{"id":1,"type":"deposit","to":1,"amount":1000,...},
{"id":2,"type":"transfer","from":1,"to":2,"amount":300,...}]
```

پیکربندی با متغیرهای محیطی

متغیر	پیش‌فرض	معنی
`PORT`	`8080`	پورت HTTP
`STORE`	`postgres`	نوع ذخیره‌سازی: `postgres` یا `memory`
`DATABASE_URL`	ندارد (وقتی `STORE=postgres` است اجباری است)	مثلاً `postgres://minibank:minibank@localhost:5432/minibank?sslmode=disable`

- با `STORE=memory` برنامه **بدون دیتابیس** و با یک store درون‌حافظه‌ای اجرا می‌شود. این حالت برای تست‌ها و شروع سریع کار است.
- اگر `STORE=postgres` باشد ولی `DATABASE_URL` تنظیم نشده باشد، برنامه باید با یک پیام واضح در لاگ **فوراً خارج شود** (fail fast).
- با سیگنال `SIGINT` یا `SIGTERM` برنامه **مرتب خاموش می‌شود**: درخواست‌های در حال اجرا را تمام می‌کند (حداکثر ۱۰ ثانیه)، اتصال دیتابیس را می‌بندد و پیامی در لاگ می‌نویسد. Kubernetes هم برای متوقف کردن pod همین `SIGTERM` را می‌فرستد.
- لاگ‌ها با `log/slog` به‌صورت JSON چاپ شوند: برای هر درخواست یک خط با `method`، `path`، `status` و `duration_ms`.

اینترفیس Store (راهنمایی)

می‌توانی از همین اینترفیس استفاده کنی. دو پیاده‌سازی لازم است: یکی درون‌حافظه‌ای با `sync.Mutex` و یکی PostgreSQL.

```go
type Store interface {
    CreateAccount(ctx context.Context, owner string) (bank.Account, error)
    ListAccounts(ctx context.Context) ([]bank.Account, error)
    GetAccount(ctx context.Context, id int64) (bank.Account, error)
    Deposit(ctx context.Context, id, amount int64) (bank.Account, error)
    Withdraw(ctx context.Context, id, amount int64) (bank.Account, error)
    Transfer(ctx context.Context, from, to, amount int64) (bank.Transaction, error)
    ListTransactions(ctx context.Context, accountID int64) ([]bank.Transaction, error)
    Ping(ctx context.Context) error
    Close()                                    // برای خاموشی مرتب
}
```

خطاهای ثابت (sentinel) پیشنهادی در پکیج `bank` : `ErrInvalidOwner` ، `ErrInvalidAmount` ، `ErrSameAccount` ، `ErrAccountNotFound` ، `ErrInsufficientFunds` .

🏆 امتیازبندی

مرحله	موضوع	امتیاز
S0	شروع: «تصمیم‌های من» (روش Q)، ریپازیتوری، جریان شاخه‌ها	۵
S1	برنامه‌ی Go: دامنه، PostgreSQL، API، هم‌روندی	۲۵
S2	تست‌ها: جدول‌محور، هم‌زمانی، HTTP	۸
S3	لینوکس و اسکریپت‌های Bash	۷
S4	Docker	۹
S5	Docker Compose	۷
S6	CI با GitHub Actions	۶
S7	Kubernetes	۱۰
S8	مستندات، یادداشت AI و تاریخچه‌ی Git	۳
	جمع ساخت	**۸۰**
D	دفاع حضوری (بدون هوش مصنوعی)	۲۰
	جمع اصلی	**۱۰۰**
B1 تا B4	امتیاز اضافه: متریک Helm، Ingress، Prometheus، انتقال ضد بن‌بست	۱۵+

جزئیات امتیاز تک‌تک کارها در **PLAN.md** آمده است.

رتبه

نمره	رتبه
۹۰ تا ۱۱۵	🏆 **افسانه (Legend)**

نمره	رتبه
۸۰ تا ۸۹	🥇 حرفه‌ای (Pro)
۷۰ تا ۷۹	🥈 سازنده (Builder)
۶۰ تا ۶۹	🥉 قبول (Pass)
زیر ۶۰	🔁 آزمون مجدد (Retake)

نمره‌ی نهایی قبل از تعیین رتبه و قبولی به نزدیک‌ترین عدد صحیح گرد می‌شود (مثلاً ۵۹٫۵ می‌شود ۶۰ و ۵۹٫۴ می‌شود ۵۹). امتیاز اضافه هم در رسیدن به نمره‌ی قبولی حساب می‌شود.

🔍 چطور تصحیح می‌شوی

۱. **در ۳:۰۰ دست از کار بکش.** برنامه‌ات را در Kubernetes در حال اجرا نگه دار و با `port-forward` روی `localhost:8080` در دسترس بگذار (اگر Compose هنوز روشن است، اول `docker compose down` بزن تا پورت 8080 آزاد شود). اگر Kubernetes کار نکرد، برنامه را با Docker Compose اجرا کن. بعد از ۳:۰۰ هیچ چیزی را در کد یا استقرار عوض نکن؛ تصحیح روی آخرین کامیت pushشده انجام می‌شود.

۲. در نوبت دفاع، مدرس تصحیح‌گر خودکار را **روی لپ‌تاپ خودت** اجرا می‌کند. این برنامه:

- همه‌ی endpointها را صدا می‌زند و ورودی خراب، مبلغ منفی، شناسه‌ی ناموجود و شناسه‌ای مثل `abc` می‌فرستد. **سرور تو هرگز نباید crash کند یا برای ورودی بد `500` برگرداند.**
- **۱۰۰ واریز هم‌زمان** و **۵۰ انتقال هم‌زمان** می‌فرستد. اگر حتی یک ریال گم شود، امتیاز هم‌روندی را از دست می‌دهی.
- ریپازیتوری، Dockerfile، فایل workflow، Compose و فایل‌های Kubernetes تو را بررسی می‌کند.

۳. **دفاع حضوری (۲۰ امتیاز، بدون هوش مصنوعی):** توضیح مسیر یک درخواست در کد خودت، یک سؤال «چرا؟» درباره‌ی تصمیم‌هایت، رفع یک خرابی که مدرس در استقرارت ایجاد می‌کند، و یک تغییر کوچک زنده در کمتر از ۳ دقیقه.

۴. **نمره‌ی نهایی** = ساخت (حداکثر ۸۰) + دفاع (حداکثر ۲۰) + امتیاز اضافه (حداکثر ۱۵).

۵. **امتیاز هم‌روندی (۳) فقط روی datastore واقعی حساب می‌شود.** اگر برنامه با `STORE=memory` اجرا شود، این ۳ امتیاز و امتیاز اضافه‌ی B4 صفر می‌شود؛ پس برنامه‌ای که تصحیح می‌شود باید به PostgreSQL وصل باشد.

> 💡 **نکته:** قبل از شروع کدنویسی، چک‌لیست امتیازها را در PLAN.md بخوان. وقتی بدانی امتیازها کجا هستند، جمع کردنشان آسان است.

🧰 یادآوری چک‌لیست آماده‌سازی (باید از قبل انجام شده باشد)

این‌ها را باید طبق «چک‌لیست آماده‌سازی» که یک هفته قبل از آزمون گرفتی، از قبل روی لپ‌تاپ خودت آماده کرده باشی. خلاصه‌اش:

- ☐ **Go** نسخه‌ی ۱.۲۶ یا جدیدتر (`go version`). در Linux و WSL، `go test -race` به کامپایلر C نیاز دارد: `sudo apt install -y build-essential` (در macOS لازم نیست).
- ☐ **Git** و حساب **GitHub** با کلید SSH که کار می‌کند (`ssh -T git@github.com`).
- ☐ **Docker** روشن باشد (`docker run --rm hello-world`).

- [ ] **kubectl** و **kind** (پیشنهادی) یا **minikube**، با کلاستری به نام `exam` که از قبل ساخته‌ای و **پاکش نکرده‌ای**. با `kubectl get nodes` باید وضعیت نود `Ready` باشد. دستور ساخت (فقط یک بار):

```
kind create cluster --name exam
```

- [ ] **curl** و **Python 3** نصب باشند (تصحیح‌گر با Python 3 روی لپ‌تاپ تو اجرا می‌شود). **jq** اختیاری است.
- [ ] ایمیج‌ها دانلود شده باشند. تگ `golang` را با نسخه‌ی Go خودت هماهنگ کن (اگر `go version` می‌گوید `go1.27`، بزن `golang:1.27-alpine`):

```
docker pull golang:1.26-alpine
docker pull postgres:16-alpine
docker pull alpine:3.22
```

- [ ] ماژول‌های Go از قبل در cache باشند (این دستورها را **داخل پوشه‌ی پروژه‌ات اجرا نکن**):

```
mkdir -p /tmp/warmup && cd /tmp/warmup && { [ -f go.mod ] || go mod init warmup; }
go get github.com/jackc/pgx/v5 github.com/prometheus/client_golang/prometheus
```

- [ ] **کاربران macOS:** ترمینال پیش‌فرض (zsh) علامت `#` را در خط فرمان کامنت حساب نمی‌کند. یک بار این را بزن و ترمینال را دوباره باز کن: `echo 'setopt interactivecomments' >> ~/.zshrc`
- [ ] **کاربران ویندوز:** همه‌ی کارها داخل WSL2 (Ubuntu) انجام شود و WSL integration در Docker Desktop روشن باشد.
- [ ] **اگر اینترنت محدود است (ایران):**
  - Docker Hub: یک `registry-mirrors` تنظیم کن (در Docker Desktop از Settings → Docker Engine؛ در Engine لینوکس در فایل `/etc/docker/daemon.json` و بعد `sudo systemctl restart docker`) یا از VPN استفاده کن. آدرس mirror را مدرس اعلام می‌کند.
  - Go: `go env -w GOPROXY=https://goproxy.io,direct`. این فقط روی سیستم خودت اثر دارد. اگر داخل `docker build` دانلود ماژول‌ها شکست خورد، در مرحله‌ی ساخت Dockerfile و قبل از `RUN go mod download` این خط را بگذار: `ENV GOPROXY=https://goproxy.io,direct`
  - کلاستر kind از mirror داکر استفاده نمی‌کند. ایمیج دیتابیس را از Docker خودت داخل کلاستر بارگذاری کن: `kind load docker-image postgres:16-alpine --name exam` (ایمیج برنامه‌ات را هم بعد از هر build همین‌طور load کن: `kind load docker-image minibank:v1 --name exam`).
  - از ایمیج‌های `gcr.io` و `registry.k8s.io` (مثل distroless) دوری کن؛ برای مرحله‌ی آخر Dockerfile از `scratch` یا `alpine` استفاده کن.

موفق باشی! یادت باشد: **بانکی که پول گم می‌کند از بانکی که کند است بدتر است.** 🐹